



## SUMÁRIO

|                                  |    |
|----------------------------------|----|
| O QUE VEM POR AÍ? .....          | 3  |
| HANDS ON .....                   | 4  |
| SAIBA MAIS .....                 | 5  |
| O QUE VOCÊ VIU NESTA AULA? ..... | 11 |
| REFERÊNCIAS.....                 | 12 |

EMSE

## O QUE VEM POR AÍ?

Nesta aula, aprenderemos o que são Large Language Models (LLMs) e como realizar sua integração de maneira eficiente utilizando o LangChain. Neste conteúdo, exploraremos as melhores práticas para criar fluxos de trabalho poderosos com esses modelos de linguagem.



## HANDS ON

Nesta aula prática, aprendemos como criar integrações utilizando o LangChain e diferentes Large Language Models (LLMs). Exploraremos os conceitos abordados nas aulas anteriores para desenvolver um código robusto, capaz de se integrar com múltiplos modelos de linguagem de forma eficiente e flexível.



## SAIBA MAIS

Começaremos a seção “Saiba mais” dessa disciplina com uma proposta importante: entender o que é um **LLM**.



Figura 1: Exemplo de fluxo de LLM  
Fonte: Tamanna (2023)

Um **Large Language Model** é um modelo de linguagem treinado em grandes volumes de texto, o que permite que ele entenda, gere e manipule a linguagem natural. Modelos como o GPT-4, da OpenAI, são exemplos de LLMs que podem realizar tarefas como:

- **Geração de texto:** produzir texto baseado em um prompt.
- **Resolução de problemas:** responder perguntas ou resolver equações.
- **Resumo:** condensar textos longos em resumos curtos e compreensíveis.
- **Tradução:** converter texto de um idioma para outro.

Os LLMs são amplamente utilizados em diversas áreas, desde assistentes virtuais até análise de dados complexos, e a integração do **LangChain** com **LLMs** é uma das suas funcionalidades mais poderosas.

Conforme aprendemos nas aulas anteriores, o LangChain permite encapsular a interação com esses modelos em "chains" modulares, facilitando a construção de pipelines que combinam diferentes tarefas em uma sequência. Isso inclui a geração de prompts personalizados, o roteamento de respostas para diferentes modelos e o encadeamento de respostas de um modelo para outro. Mas você sabia que também existem outros modelos de LLMs além dos populares Gemini e GPT?

Embora modelos como **GPT** da OpenAI e **Gemini** do Google sejam amplamente conhecidos, existem diversos outros **Large Language Models (LLMs)** que também desempenham papéis importantes no campo da IA.

Cada um desses modelos tem características únicas, treinamentos específicos e pode ser mais adequado para determinadas tarefas ou tipos de dados. Vamos conhecer alguns deles:

- **YOLO (You Only Look Once)**: um modelo usado para detecção de objetos em tempo real. Ele é capaz de detectar múltiplos objetos em uma imagem com alta velocidade e precisão.
- **O Mistral 7B**: é um modelo desenvolvido por Mistral AI. Ele é um exemplo de um modelo de linguagem pré-treinado, projetado para processar e gerar texto natural de alta qualidade.
- **LLama (Large Language Model Meta AI)**: criado pela Meta, o LLama foi desenvolvido para ser mais eficiente em termos de tamanho e performance, sendo utilizado em cenários que exigem o processamento de grandes quantidades de dados com alta precisão.
- **Claude (Anthropic)**: Claude é um modelo desenvolvido com foco na segurança e previsibilidade de suas respostas. Ele é otimizado para minimizar erros e vieses, sendo uma alternativa interessante para aplicações que exigem um alto nível de confiabilidade.
- **Bloom**: um modelo de código aberto, desenvolvido em colaboração internacional e que é conhecido por sua diversidade linguística, suportando

várias línguas e permitindo que desenvolvedores(as) tenham maior controle sobre seu comportamento.

- BERT (Bidirectional Encoder Representations from Transformers): desenvolvido pela Google, o BERT é um dos primeiros modelos de linguagem baseado em transformadores. Ele é conhecido por seu foco em tarefas de compreensão de linguagem natural, como a classificação de texto e a resposta a perguntas.
- T5 (Text-To-Text Transfer Transformer): outro modelo da Google, o T5 foi treinado para transformar várias tarefas de processamento de linguagem em um problema de conversão de texto. Isso o torna extremamente versátil, tornando-o capaz de resolver desde tarefas de tradução até a sumarização de textos.

Esses são apenas alguns exemplos de LLMs que, assim como o GPT e o Gemini, desempenham papéis fundamentais na IA. A escolha do modelo mais adequado depende do contexto da aplicação, da quantidade de dados e dos requisitos de desempenho. Ao utilizar o LangChain, você pode integrar vários desses modelos, escolhendo o mais eficiente para cada tarefa específica.

Agora que sabemos o que é um LLM e vimos outros exemplos, vejamos como integrar um LLM com o LangChain.

## Integração LLM com o LangChain

O primeiro passo é definir qual LLM será usado. O **LangChain** oferece suporte a diversos modelos, como o **GPT-4** da OpenAI (que nós vimos em aulas anteriores), além de outros LLMs disponíveis no mercado.

Neste exemplo, vamos utilizar o **modelo LLaMA2** integrado ao **LangChain** por meio da biblioteca **Ollama**. O código faz a análise de sentimentos e gera um resumo em português para textos carregados de um arquivo. Utilizamos **Chains** para encadear a execução dos prompts personalizados para cada tarefa, como a análise de sentimento e a geração de resumos.

A seguir temos o texto que analisaremos:

noticias.txt

Notícia 1: A economia global está se recuperando após a pandemia.  
Notícia 2: O mercado de ações caiu drasticamente na última semana.  
Notícia 3: Novas tecnologias estão impulsionando a inovação em diversas indústrias.

E o código:

```
from langchain_community.llms import Ollama
from langchain.callbacks.manager import CallbackManager
from langchain.callbacks.streaming_stdout import StreamingStdOutCallbackHandler
from langchain.prompts import PromptTemplate
from langchain.chains import LLMChain
from langchain_community.document_loaders import TextLoader
import os

def carregar_documentos(caminho_arquivo):
    loader = TextLoader(caminho_arquivo)
    documentos = loader.load()
    return documentos

def limpar_texto(texto):
    return texto.strip()

llm = Ollama(
    model="llama2",
    num_gpu=0,
    callback_manager=CallbackManager([StreamingStdOutCallbackHandler()])
)

prompt_sentimento = "Analisar o sentimento do seguinte texto em português: {text}"
prompt_resumo = "Gere um resumo em português para o seguinte texto: {text}"
```

```

template_sentimento = PromptTemplate(input_variables=["text"],
template=prompt_sentimento)
template_resumo = PromptTemplate(input_variables=["text"],
template=prompt_resumo)

chain_sentimento = LLMChain(llm=llm, prompt=template_sentimento)
chain_resumo = LLMChain(llm=llm, prompt=template_resumo)

caminho_arquivo = "noticias.txt"

if not os.path.exists(caminho_arquivo):
    raise FileNotFoundError(f"O arquivo {caminho_arquivo} não foi encontrado.")

documentos = carregar_documentos(caminho_arquivo)

for doc in documentos:
    texto_limpo = limpar_texto(doc.page_content)

    resultado_sentimento = chain_sentimento.invoke({"text": texto_limpo})
    resultado_resumo = chain_resumo.invoke({"text": texto_limpo})

    print(f"Notícia: {texto_limpo}")
    print(f"Sentimento: {resultado_sentimento}")
    print(f"Resumo: {resultado_resumo}")
    print("-" * 120)

```

Note que neste código nós **carregamos documentos de texto**, utilizamos o **modelo LLaMA2** para analisar o sentimento e gerar um resumo em português, e organizamos o fluxo através de **Chains** no LangChain. Além disso, limpamos o conteúdo dos textos antes de processá-los e exibimos o resultado, que inclui a notícia, o sentimento identificado e o resumo gerado.

Agora atualizaremos o nosso código para utilizar uma outra LLM. Como estamos utilizando o Ollama, podemos escolher outra model na [biblioteca](#). Neste

exemplo, atualizaremos o nosso código para utilizar o mistral. Para isso, basta atualizar o seguinte trecho de código:

```
llm = Ollama(  
    model="mistral",  
    num_gpu=0,  
    callback_manager=CallbackManager([StreamingStdOutCallbackHandler()])  
)
```

Notem como a utilização do LangChain facilita bastante esta integração.

## O QUE VOCÊ VIU NESTA AULA?

Nesta aula, aprendemos o que são Large Language Models (LLMs) e como realizar sua integração de maneira eficiente utilizando o LangChain, além de explorar as melhores práticas para criar fluxos de trabalho poderosos com esses modelos de linguagem.

EMANDA

## REFERÊNCIAS

LLMs. **langchain**. [s.d.]. Disponível em: <https://python.langchain.com/v0.2/docs/integrations/llms/>. Acesso em: 09 set. 2024.

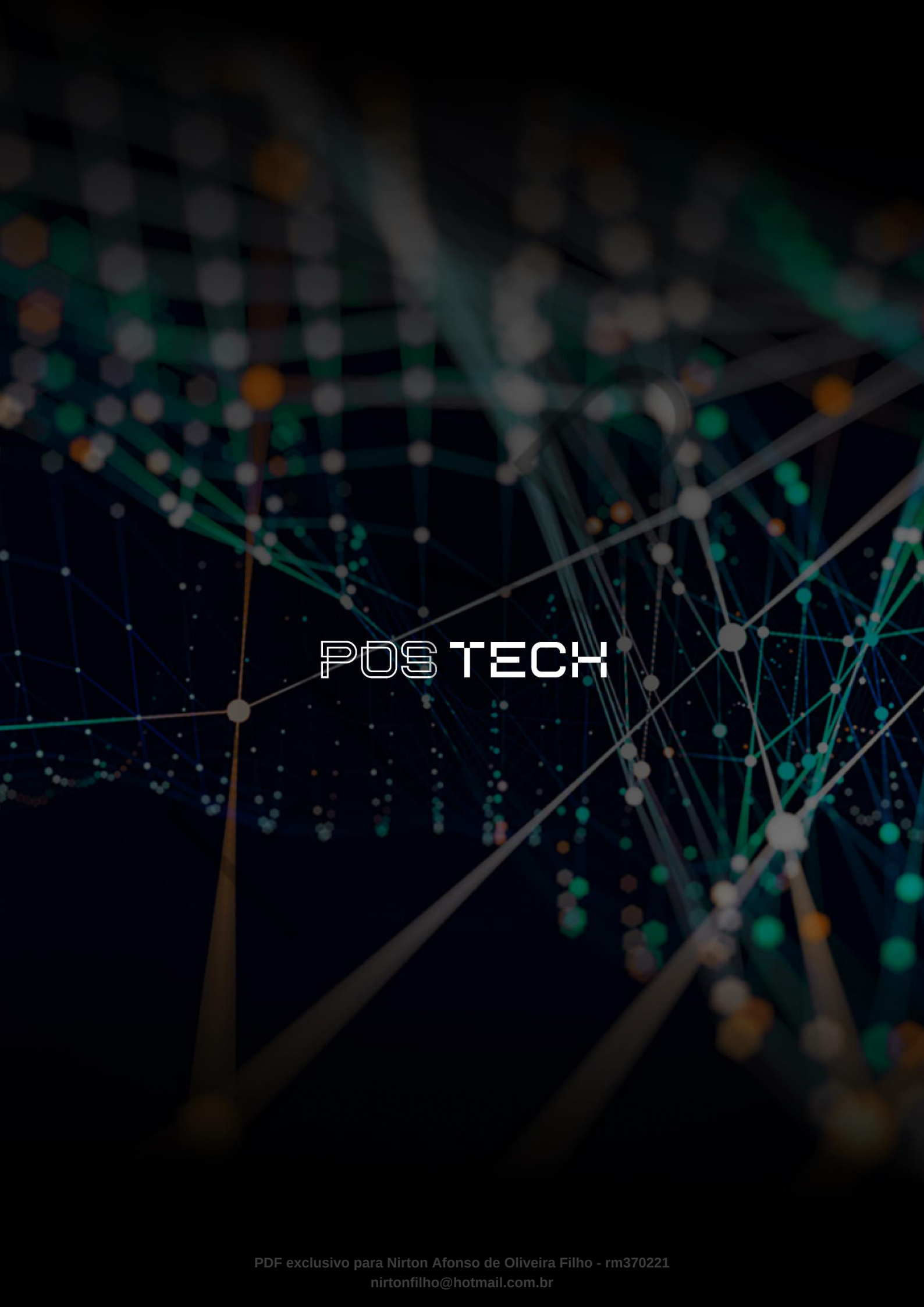
MORAES, J. Large Language Models: o que são e como aplicá-los no dia a dia. **coursespm3**. 2024. Disponível em: <https://www.coursespm3.com.br/blog/large-language-models/>. Acesso em: 09 set. 2024.

PEREIRA, W. Como IA e LLMs revolucionam a interação entre varejistas e clientes. **sensedia**. 2023. Disponível em: <https://www.sensedia.com.br/post/como-ia-e-llms-revolucionam-a-interacao-entre-varejistas-e-clientes>. Acesso em: 09 set. 2024.

## **PALAVRAS-CHAVE**

**Palavras-chave:** LangChain. LLMs. Ollama. Model.

EMSE



POSTECH